

R2 自动机器人：ROS 2 与 STM32 高频双工协议开发指南

技术栈：ROS 2 Control (Humble) & STM32 HAL库 — 专为电控与上位机解耦设计

一、模块简介与设计架构

在 R2 自动机器人等复杂嵌入式系统中，上层行为决策（BehaviorTree/MoveIt 2）与底层电机控制（CAN总线/达妙电机驱动）之间的数据同步频率要求极高。传统的单字节串口接收中断方案会使 CPU 频繁挂起，导致严重的锁竞争、丢包甚至是总线死锁。现象通常表现为：“上位机下发控制正常，但下位机状态无法成功上报”。

本协议模块采用“后台 DMA 循环接收（Circular Mode）”+“主循环状态机异步解包”+“事件驱动发送保护”的全新架构，实现了通信层与运动控制算法层的完全解耦。其核心精髓在于“数据黑板模型”：

🔥 数据黑板交互机制

电控业务开发者无需关心串口何时产生粘包、如何对齐帧头、何时计算校验。开发者仅需要做两件事：

1. **取指令**：从只读货架 `g_target_angles` 中读取上位机最新的弧度目标。
2. **填状态**：将电机的编码器真实反馈实时填入 `g_actual_angles` 黑板，底层会自动将其高频上报。

二、STM32CubeMX 核心配置步骤

为确保底层硬件加速管道畅通，必须严格在 STM32CubeMX 中对 `USART2` 进行如下配置：

1. 串口基础参数 (Parameter Settings)

- **Baud Rate (波特率)**: `115200` (须与上位机 `arm_hardware_interface.cpp` 严格一致)
- **Word Length (字长)**: `8 Bits`
- **Parity (校验)**: `None`
- **Stop Bits (停止位)**: `1`
- **Mode (模式)**: `Asynchronous` (异步双工)，开启 `TX` 和 `RX`

2. DMA 传输配置 (DMA Settings) — 核心要点

点击 **Add** 按钮，添加 `USART2_RX` 和 `USART2_TX` 两条硬件流。参数必须按表配置：

DMA 请求类型	方向 (Direction)	优先级	模式 (Mode)	外设自增 (Periph Inc)	内存自增 (Mem Inc)	数据宽度
USART2_RX	Peripheral To Memory	Medium / Low	Circular (循环模式)	DISABLE (不自增)	ENABLE (自增)	Byte
USART2_TX	Memory To Peripheral	Low	Normal (正常模式)	DISABLE (不自增)	ENABLE (自增)	Byte

⚠ 为什么 RX 必须选择 Circular，而外设不能自增？

- 1. 外设不自增：**串口接收寄存器 `USART2->DR` 的物理地址是固定的，DMA 每次必须去同一个地方拿水。如果误开自增，指针跑飞会导致 HardFault。
- 2. RX 选择 Circular：**DMA 会在后台将数据源源不断地搬运到环形数组里，写满一圈自动移回头部覆盖旧数据，全程零 CPU 干预。若选择 Normal，缓冲区满一次 DMA 就会关闭，导致单片机“永久失聪”。

3. 中断配置 (NVIC Settings)

- 勾选 `USART2 global interrupt` 开启全局中断。
- 确保两条 DMA Stream 的 `global interrupt` 处于勾选开启状态（HAL库需要通过发送完成中断来恢复 `gState` 到 READY 状态）。

三、下位机业务代码使用规范

重新生成代码后，该协议模块向外部导出了极其精简的 API 和变量接口。电控开发者需要在 `main.c` 和控制闭环里落实以下标准操作：

1. 初始化与主循环轮询

在 `main.c` 中，必须将初始化置于外设配置之后，并在主循环里进行无阻塞轮询：

```
int main(void)
{
    /* 1. 硬件外设初始化 */
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_CAN1_Init();
    MX_USART2_UART_Init();

    /* USER CODE BEGIN 2 */
    Protocol_Init(); // 2. 初始化协议黑板，并一键启动底层硬件 DMA 环形接收管道
    Can_Filter_Init();
    DMmotor_init_all();
    /* USER CODE END 2 */

    /* 主循环轮询 */
    while (1)
    {
        Protocol_Process(); // 3. 通信核心状态机轮询：静默接收并自动执行100Hz定时状态上报

        all_motor_ctrl(); // 4. 执行业务电机控制
    }
}
```

2. 提取下发指令 (ROS 2 -> 电机)

为防止单片机极高频的 `while(1)` 产生控制洪泛，瘫痪 CAN 总线，必须采用“事件驱动机制”。判断 `g_target_updated` 为 1 时才允许提取并控制电机，用完必须立刻清零：

```

/* 在电机控制文件顶部或 main.c 中声明接口 */
extern float g_target_angles[MAX_JOINTS]; // 只读目标角度数组（弧度制）
extern uint8_t g_target_updated;          // 指令更新标志位

void all_motor_ctrl(void)
{
    // 核心拦截器：只有当收到了上位机 100Hz 的全新下发序列时才触发动作
    if (g_target_updated == 1)
    {
        g_target_updated = 0; // 【关键】：提取前必须立刻手动清零！

        // 提取数据（数组下标 0~2 对应真实硬件 ID 1~3 的物理关节电机）
        float joint1_target = g_target_angles[0];
        float joint2_target = g_target_angles[1];
        float joint3_target = g_target_angles[2];

        // 喂给你的达妙电机控制驱动函数（示例：控制位置模式）
        DMmotor_ctrl_position(&motor[0], joint1_target);
        DMmotor_ctrl_position(&motor[1], joint2_target);
        DMmotor_ctrl_position(&motor[2], joint3_target);
    }
}

```

3. 上报物理电机姿态 (下位机 -> ROS 2)

为使上位机 Rviz 模型和 MoveIt 能感知物理实体机械臂的当前位置，必须在 CAN 接收回调里实时覆写 `g_actual_angles` 状态黑板：

```

extern float g_actual_angles[MAX_JOINTS]; // 状态上报黑板

void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan)
{
    if (hcan->Instance == CAN1)
    {
        CAN_RxHeaderTypeDef rx_header;
        HAL_CAN_GetRxMessage(&hcan1, CAN_RX_FIFO0, &rx_header, rx_data);

        uint8_t motor_id = (rx_data[0]) & 0x0F; // 提取反馈帧中的电机ID

        if (motor_id >= 1 && motor_id <= 3)
        {
            motor_t *m = &motor[motor_id - 1];

            DMmotor_fbdata(m, rx_data); // 调用驱动解析出当前电机的真实弧度位置 m->pos

            // 【同步更新至黑板】：Protocol_Process 会自动在下个10ms内利用 TX DMA 盲发给 ROS 2
            g_actual_angles[motor_id - 1] = m->pos;
        }
    }
}

```

四、极端异常自愈与灾备机制

在 Robocon 赛场复杂的强电磁干扰环境下，高频串口链路极易产生单字节毛刺，引发 **Overrun Error (ORE, 溢出错误)** 或 **Frame Error (FE, 帧错误)**。一旦触发，HAL库底层会保护性地**彻底关闭 DMA 接收流**，产生假死。本架构集成了一键满血复活的异常自愈机制。

1. 异常复位接口：Protocol_On_UART_Error()

由于改用后台环形 DMA，当硬件发生 Abort 强制中止时，底层的硬件计数器会突变回最大值。此时，必须同步对软件层的环形读写指针以及状态机进行清零，否则会导致数据流指针错位、解析出致命乱码指令。该函数已被封装：

```
void Protocol_On_UART_Error(void)
{
    // 1. 强行同步复位环形队列的读写指针，匹配重启后的硬件寄存器
    rx_head = 0;
    rx_tail = 0;

    // 2. 强制复位状态机当前状态与计数，扔掉历史残缺帧，等待全新的 0xAA 0x55 帧头
    rx_state = 0;
    rx_frame_idx = 0;
}
```

2. 强鲁棒性错误回调集成

电控人员必须在 **main.c** 尾部的 **HAL_UART_ErrorCallback** 中进行如下规范编写。一旦硬件报错，它将作为“自动除颤仪”在微秒级内拉起全套管道：

```
void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2)
    {
        // 1. 强制中止当前死锁的串口和接收流，解锁 HAL 库内部的锁状态
        HAL_UART_AbortReceive(huart);

        // 2. 调用协议模块的复位接口，优雅清除软件环形指针与残缺状态机
        Protocol_On_UART_Error();

        // 3. 重新激活后台无感 DMA 循环接收通道，瞬间恢复双工通信
        HAL_UART_Receive_DMA(&huart2, (uint8_t *)rx_ring_buf, RX_RING_BUF_SIZE);
    }
}
```

运维调测建议

- **单字节中断清理**：如果你之前在 `HAL_UART_RxCpltCallback` 里面写了针对 `USART2` 的单字节接收代码（如 `Protocol_UART_RxCallback()`），**请务必全部注释/删除**，现在已跨入全硬件 DMA 时代，无需让 CPU 进中断处理单字节。
- **严禁长延时阻塞**：因为 DMA 环形缓冲区大小 `RX_RING_BUF_SIZE` 通常较小（如 128/256 字节），主循环中严禁调用类似于 `HAL_Delay(50)` 这样的阻塞函数。一旦主循环卡死超过 50ms，会导致 DMA 倒水过快直接没过 `rx_tail` 发生“套圈套死”溢出。请使用基于 `HAL_GetTick()` 的增量时间片进行业务逻辑的分时复用。